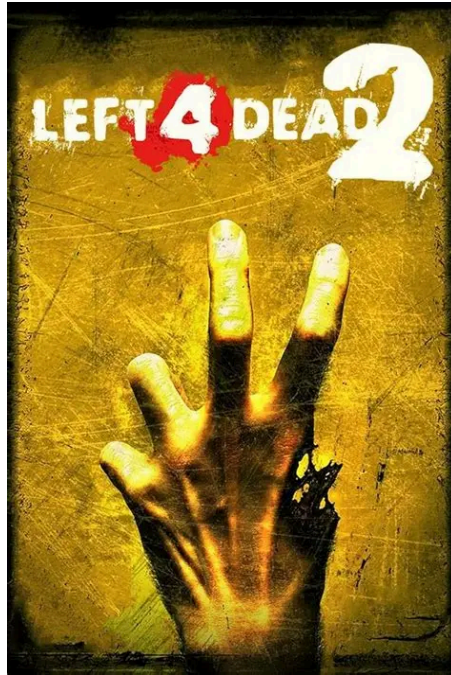


Course: Individual Software Development Process 2026

Iteration Report: Iteration 3

By Left4Dead

Chosen Irl Challenge: Project D: Course Support & Activity Dashboard



Thanawin Thanapornthawan	6810545654
Phutarak Wongpitak	6810545841
Pakhin Daonan	6810545859
Kantanat Utamapongchai	6810545468

Table of Contents

1. Project Snapshot.....	1
1.1 Project-Wide Gantt Chart.....	1
1.2 Current Iteration Gantt Chart.....	2
2. Sprint/Board Status Snapshot.....	2
3. Scope Addition/Change Log.....	4
4. Retrospective.....	5
5. Testing & Quality Notes.....	5
6. Project Risk Register.....	6
7. Individual Contribution Log.....	6

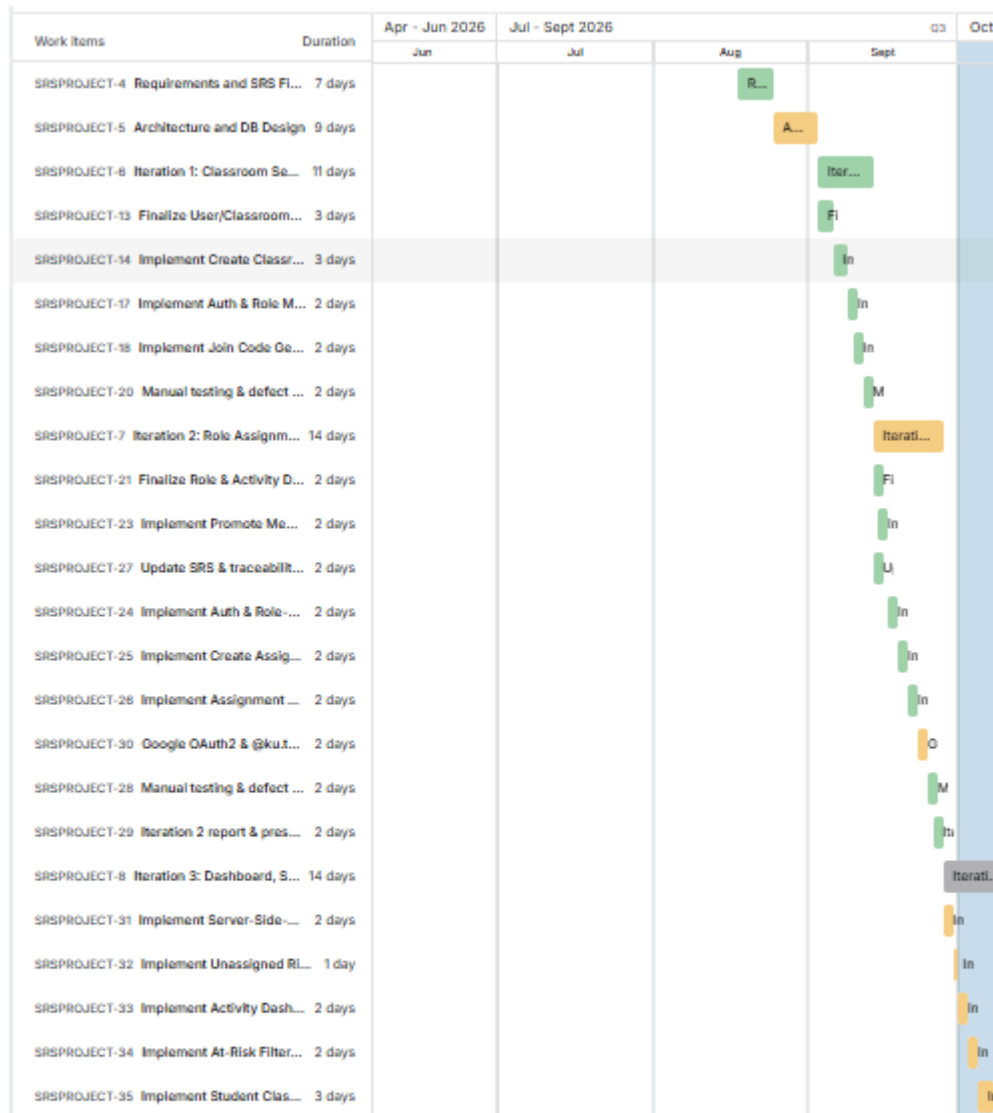
1. Project Snapshot

Note: This section must include both the Gantt chart for the whole project and the calculation of the critical path for the project and each iteration. Created via Plane.so.

Note: Project Repository is mandatory for the report submission of Iteration 2. For Iteration 1, this can be left blank as "(WIP)".

Project Repository: [ThanawinTH/ISP-Left4Dead](#)

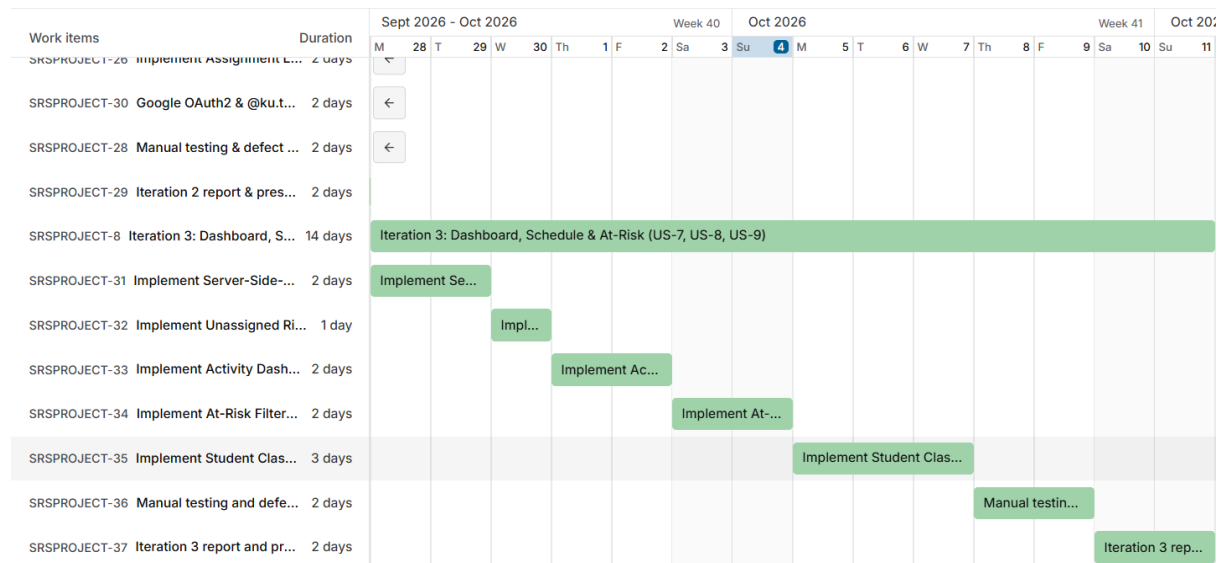
1.1 Project-Wide Gantt Chart



Critical Path: Requirements and SRS Finalization → Architecture and DB Design → Iteration 1: Classroom Setup & Membership (US-1, US-2, US-3) → Iteration 2: Role Assignment, Activity Management, Google OAuth2 & @ku.th Restriction (US-4, US-5, US-6, US-16) → Iteration 3: Dashboard, Schedule & At-Risk (US-7, US-8, US-9) → Iteration 4: Announcements & Staff Channel (US-10–US-13) → Iteration 5: File Submission & Status (US-14, US-15) → Iteration 6: Docker, Testing & Deployment

Critical Path Duration: 98 calendar days

1.2 Current Iteration Gantt Chart



Critical Path: Implement Server-Side-At-Risk Classification → Implement Unassigned Risk Rule → Implement Activity Dashboard Ordered by Due Date → Implement At-Risk Filtered List → Manual testing and defect fixes → Iteration 3 report and presentation

Critical Path Duration: 14 calendar days

2. Sprint/Board Status Snapshot

Note: User stories must be consistent with the user stories in Software Proposal (Week 4). For Maximum Capacity, It Must Be:

- 3 User Stories for Iteration 1
- 4 User Stories for Iteration 2

Iteration Goal: Deliver a working role assignment and activity management slice so that a lecturer can promote classroom members to TA or university staff, while authorized staff can create activities and update their status. This iteration directly supports Sub-Goals SG-1 and SG-2 from the SRS.

Capacity Declared: 4 user stories (maximum capacity for Iteration 2).

User Story	Description	Owner	Status	Within Capacity (Y/N)	Notes
US-7	As a lecturer or TA, I want one integrated view of upcoming activities so that I do not have to check several tools.	Phutarak Wongpitak , Thanawin Thanaporn thawan	Done	Y	The backend returns the classroom's activities ordered by due date, with the owner or TAs, visibility, and status. The front end implements the dashboard columns. Covers SRS-13

US-8	As a lecturer or TA, I want at-risk activities highlighted so that nothing is missed.	Pakhin Daonan, Thanawin Thanapornthawan	Done	Y	The backend implements the risk classification in SQL (Overdue, Due Soon, Unassigned) Covers SRS-12 and SRS-13.
US-9	As a student, I want to view the class schedule so that I know when sessions and deadlines fall.	Phutarak Wongpitak, Kantanut Utamapongchai	Done	Y	The backend filters the list to Class-visibility activities for student accounts and rejects any create, edit, or delete request from a student. Covers SRS-14.

3. Scope Addition/Change Log

Note: Log any item added, expanded, or swapped in after planning began. Empty is a good sign; frequent entries signal scope creep.

Item added/changed	Reason	What was dropped to compensate (one-in-one-out)?
The backend half of US-9 (showing students only class-visibility activities and rejecting any create, edit or delete request from a student account) was already in place before Iteration 3 began. It was written during Iteration 2 inside the US-5 work.	The assignment list built in Iteration 2 had to behave correctly for a student as soon as it existed, so the visibility rule could not wait for its own iteration.	None. The work was already counted inside Iteration 2. Iteration 3 added only the student schedule page and checked the behaviour against SRS-14.
US-8 was built for the student view first. The classification the server returns is Closed, Due Soon or OK, and only a student account receives it	The student schedule page needed the risk field in order to show work that is due soon, so the team built the shared SQL classification there first and reused it.	None was dropped from this iteration. The staff-side highlight described in the user story moves to Iteration 4.

The Unassigned rule in SRS-12 (an activity with no owner whose due date falls within 7 days) was not implemented in this iteration.	The view that was built is the student schedule, and a student never sees an activity's owner, so the rule would have had no visible effect on that page.	None. The rule is carried to Iteration 4 and will be delivered together with the staff-side at-risk view that makes it visible.
---	---	---

4. Retrospective

What went well	What didn't go well	Action items for next sprint
The team divided the work clearly between backend and frontend. Each member knew what they were responsible for in US-7, US-8, and US-9.	US-16 (Google OAuth2) is still in progress. The team focused on the dashboard, at-risk, and schedule features first.	Try to finish learning and test US-16 early in Iteration 4
The team followed the SRS and user stories when developing the features, including the at-risk rules and the student visibility rule.	The at-risk feature was hard to test because it depends on the current date. Some cases were only checked near the end.	Prepare test data for every at-risk case before starting development.
The team worked together and reused the same list for the dashboard and the at-risk view, so less work was repeated.	The frontend had to wait for the backend to be ready, so the integration happened late in the sprint.	Agree on the API response format first, so frontend and backend can work at the same time.

5. Testing & Quality Notes

Note: any bugs, defects, tests (informal & formal) encountered and/or fixed during this iteration can be included here. If there is none, just put "(None applicable)" below.

- **Students see "Submitted" for assignments they have not submitted.** The page was using the wrong status information. Fixed by using the correct student submission status and retested the affected case.

- **Activities with the same due date can change order between page loads.** The system did not have a consistent secondary ordering. Fixed by adding a secondary ordering and tested again.

6. Project Risk Register

Risk ID	Description	Likelihood (L/M/H)	Impact (L/M/H)	Mitigation/Contingency	Status
R-01	Some tasks depend on other tasks and may cause delays.	M	M	Start important tasks early and tell the team when blocked.	Open
R-02	Frontend and backend may not work together correctly.	M	H	Test the API with the frontend early. Agree on the response format before implementation.	Open
R-03	The at-risk classification depends on the current date, so wrong results may go unnoticed without proper test data.	M	M	Prepare dated test data for every at-risk case and test the boundary cases early.	Open

7. Individual Contribution Log

Note: One row per member, filled in by that member. The submission of this file considered that the form below is consented and confirmed by the whole team. This section directly supports the Individual Project Contribution and Peer Evaluation grading components.

Note 2: For Iteration 1, evidence can be a reference to the video presentation/iteration report that is related to the responsible task(s) (e.g. Lalita's Evidence). However, for Iteration 2, the reference must be on **GitHub file and commit only, which is uploaded by that team member**.

Student Name	Tasks owned this sprint	Course concept(s) applied	Evidence	Self-rated effort (1-5)
Pakhin Daonan	US-8 backend: at-risk classification in SQL and ?atRisk=1 filter (SRS-12, SRS-13)	User story to SRS traceability; acceptance testing with dated test data for each risk case	6969297 US-8: compute student at-risk in SQL and add ?atRisk=1 filter	5
Thanawin Thanapornthawan	US-7 dashboard columns; US-8 due-soon box and toggle	Separation of concerns (server decides, client displays); API contract between front end and back end	a258371 US-7 dashboard columns; 7d1cef8 US-8: at-risk box and at-risk-only toggle on the student schedule	5
Phutarak Wongpitak	US-7 assignment list API; US-9 lecturer-only delete (SRS-13, SRS-14)	Role-based access control enforced on the server; RESTful API design	9f709d4 US-7 assignment list; 290cc4b US-9: lecturer-only delete; students refused with 403	5
Kantanut Utamapongchai	US-9 student schedule, Delete button, acceptance test (SRS-14)	Acceptance testing against SRS requirements; manual testing and defect fixing	68a9b79 US-9: lecturer Delete button works, with a confirm	5